HTML

```
<!DOCTYPE html><html><head><title>Page</title></head><body>
<form method="POST" action="">          <!-- "" = same file -->
  <label for="nm">Name:</label>
  <input type="text" id="nm" name="nm">
  <input type="password" name="pw"> <input type="email" name="em">
  <input type="number" name="n" step="any">
  <input type="radio" name="area" value="Urban"> Urban
  <input type="checkbox" name="sub[]" value="PHP"> PHP
  <select name="city"><option value="Dhaka">Dhaka</option>
  </select>
  <textarea name="note"></textarea>
  <input type="submit" name="go" value="Submit"></form>
<button onclick="calc()">Go</button>   <div id="out"></div>
<table border="1"><tr><th>Name</th></tr>
  <tr><td>Rahim</td></tr></table>
<script src="script.js"></script></body></html>
```

id → JavaScript · **name** → PHP ($_POST)

JAVASCRIPT

Input, output, functions

```
let total = 0;  const GOAL = 20;  // outside: kept between clicks
function calc() {                 // <button onclick="calc()">
  total += parseFloat(document.getElementById("hrs").value);
  document.getElementById("out").innerHTML = `Total: ${total}`;
}
parseInt("4")  parseFloat("2.5")  Number("7")  (2.456).toFixed(2)
isNaN(x)  alert("Hi")  console.log(x)  prompt("Age?") (a string)
```

Loops

```
for (let i = 1; i <= 10; i++) { sum += i; }
while (n > 0) { n = Math.floor(n / 10); }
do { k++; } while (k < 3);              // runs at least once
for (let v of arr) { if (v < 0) continue; if (v > 99) break; }
```

Arrays

```
let a = [12, 15, 14];  a.push(7)  a.pop()  a.length  a[0] = 9
a.includes(15)  a.indexOf(15) (-1 = none)  a.join(", ")
Math.max(...a)  Math.min(...a)  a.reduce((s, v) => s + v, 0)
a.filter(v => v >= 40)  a.map(v => v * 2)
```

```
a.sort()                  // as TEXT: [10, 9, 100] -> [10, 100, 9]
a.sort((x, y) => x - y)  // numbers ascending
a.sort((x, y) => y - x)  // numbers descending
```

Strings

```
s.length  s.trim()  s.toLowerCase()  s.toUpperCase()
s.includes("ab")  s.indexOf("a")  s.slice(0, 3)  s.substring(1, 4)
s.replace("a", "b")  s.split(",")  s.split("").reverse().join("")
s.endsWith("*")  /[A-Z]/.test(s)  /[0-9]/.test(s)  /[!@#]/.test(s)
```

Math and random

```
Math.round(4.5) 5  Math.floor(4.9) 4  Math.ceil(4.1) 5
Math.abs(-3) 3  Math.sqrt(16) 4  Math.pow(2, 3) 8
Math.floor(Math.random() * (max - min + 1)) + min  // min..max
```

PHP

```
$name = "Karim";  $n = 5;  echo "Hi $name";  echo 'Hi ' . $name;
(int) $x  (float) $x  intdiv(7, 2)  7 % 2  round($x, 2)  ceil($x)
floor($x)  abs($x)  number_format(15000) -> "15,000"  rand(1, 6)
for ($i = 1; $i <= 10; $i++) { }   while ($n > 0) { }
$a = [3, 1, 2];  $a[] = 4;   $m = ["Dhaka" => 7.5];
foreach ($a as $v) { }   foreach ($m as $key => $val) { }
```

Functions

```
count($a)  array_sum($a)  max($a)  in_array(4, $a)  implode()
sort/rsort values   asort/arsort keep keys   ksort/krsort by key
strlen  strtolower  strtoupper  trim  strrev  substr($s, 0, 3)
strpos($s, "x")  str_replace("a", "b", $s)  explode(",", $s)
isset($x)  empty($x)  preg_match('/[A-Z]/', $s)
```

Reading a POST form

```
<?php
if (isset($_POST["go"])) {              // after the form posts
  $n = (int) $_POST["n"];  $nm = trim($_POST["nm"] ?? "");
  echo htmlspecialchars($nm) . ": $n";   // safe output
}
?>
```

PHP + MYSQLI

XAMPP: start Apache + MySQL · `localhost/phpmyadmin` (SQL tab: run CREATE / INSERT) · save in `htdocs/`, open `http://localhost/file.php` · user `root`, password "".

```
<?php
$conn = new mysqli("localhost", "root", "", "uiu");
if ($conn->connect_error) die("Failed: " . $conn->connect_error);
$res = $conn->query("SELECT name, cgpa FROM students");
echo "<table border='1'><tr><th>Name</th><th>CGPA</th></tr>";
while ($row = $res->fetch_assoc())      // $res->num_rows
  echo "<tr><td>" . htmlspecialchars($row["name"]) .
       "</td><td>" . $row["cgpa"] . "</td></tr>";
echo "</table>";
// a value the user typed -> prepared statement
$stmt = $conn->prepare("SELECT * FROM students WHERE city = ?");
$stmt->bind_param("s", $_POST["city"]);  // s text i int d decimal
$stmt->execute();
$res = $stmt->get_result();    // then the same fetch_assoc() loop
$conn->close();
```

SQL (EXAMPLE SCHEMA USED BY EVERY QUERY)

Create and insert

```
CREATE DATABASE uiu;
USE uiu;
CREATE TABLE departments (
  dept  VARCHAR(10) PRIMARY KEY,
  dname VARCHAR(60) NOT NULL
);
CREATE TABLE students (
  id      INT AUTO_INCREMENT PRIMARY KEY,  -- 1, 2, 3 automatic
  name    VARCHAR(50) NOT NULL,
  dept    VARCHAR(10),
  cgpa    DECIMAL(3,2),                    -- exact, e.g. 3.75
  credits INT,  city VARCHAR(30),
  joined  DATE,                            -- '2024-01-31'
  FOREIGN KEY (dept) REFERENCES departments(dept)
);
INSERT INTO departments VALUES
  ('CSE', 'Computer'), ('EEE', 'Electrical'), ('CE', 'Civil');
INSERT INTO students (name, dept, cgpa, credits, city, joined)
VALUES ('Karim', 'CSE', 3.75, 90, 'Dhaka', '2024-01-10'),
       ('Rahim', 'EEE', 2.95, 60, NULL, '2024-02-03'),
       ('Nadia', 'CSE', 3.10, 45, 'Khulna', '2024-01-25');
-- text with an apostrophe: 'Cox''s Bazar'
ALTER TABLE students ADD email VARCHAR(60);    -- add column
```

Select and filter

```
SELECT name, cgpa FROM students WHERE cgpa >= 3.5;  -- * = all
SELECT DISTINCT city FROM students;             -- each once
SELECT name, credits * 1500 AS fee FROM students;  -- new col
... WHERE city = 'Dhaka' AND cgpa > 3           -- both true
... WHERE city = 'Dhaka' OR city = 'Khulna'     -- either
... WHERE NOT city = 'Dhaka'                    -- opposite
... WHERE dept IN ('CSE', 'EEE')                -- in a list
... WHERE credits BETWEEN 45 AND 75             -- 45..75 incl.
... WHERE joined BETWEEN '2024-01-01' AND '2024-01-31'
... WHERE name LIKE 'K%'                        -- starts with K
... WHERE name LIKE '%im%'                      -- contains im
... WHERE name LIKE '_a%'                       -- 2nd letter a
... WHERE city IS NULL                          -- empty
... WHERE city IS NOT NULL                      -- has a value
```

Sort and limit

```
SELECT * FROM students ORDER BY cgpa DESC;      -- high to low
SELECT * FROM students ORDER BY dept ASC, cgpa DESC;
SELECT * FROM students ORDER BY cgpa DESC LIMIT 3;  -- top 3
```

Count, sum, average, group

```
SELECT COUNT(*) FROM students;                  -- rows
SELECT SUM(credits), ROUND(AVG(cgpa), 2), MIN(cgpa), MAX(cgpa)
FROM students;
SELECT dept, COUNT(*) AS total
FROM students GROUP BY dept;                    -- 1 row per dept
SELECT dept, AVG(cgpa) AS avg_cgpa
FROM students
WHERE credits >= 45            -- WHERE: filters ROWS first
GROUP BY dept
HAVING AVG(cgpa) > 3.0         -- HAVING: filters GROUPS after
ORDER BY avg_cgpa DESC;
SELECT city, COUNT(*) FROM students
GROUP BY city HAVING COUNT(*) > 1;              -- more than 1
SELECT dept, COUNT(DISTINCT city) FROM students GROUP BY dept;
```

Join

```
SELECT s.name, d.dname
FROM students s INNER JOIN departments d
ON s.dept = d.dept;                        -- matching rows only
SELECT d.dname, COUNT(s.id) AS total
FROM departments d LEFT JOIN students s
ON s.dept = d.dept                         -- every department
GROUP BY d.dept, d.dname;                  -- CE shows 0
```

Update and delete

```
UPDATE students SET cgpa = 3.80, city = 'Dhaka' WHERE id = 1;
UPDATE students SET credits = credits * 1.10   -- +10%
WHERE dept = 'CSE';                            -- * 0.90 = -10%
DELETE FROM students WHERE id = 2;    -- rows (no WHERE = ALL)
TRUNCATE TABLE students;              -- empty it, keep table
DROP TABLE students;                  -- the table itself goes
```

Exam patterns

```
-- add 5 only if the result stays <= 90
UPDATE students SET credits = credits + 5
WHERE credits > 80 AND credits + 5 <= 90;
-- two rules in one pass, so no row is changed twice
UPDATE students SET cgpa = CASE
  WHEN cgpa > 3.5 THEN cgpa - 0.1 ELSE cgpa + 0.1 END;
-- change only the newest row
UPDATE students SET credits = credits + 3
ORDER BY joined DESC LIMIT 1;
-- label every row
SELECT name, CASE WHEN cgpa >= 3.5 THEN 'Top'
  ELSE 'Regular' END AS label FROM students;
-- above the average of its own group (correlated subquery)
SELECT name FROM students s WHERE cgpa >
  (SELECT AVG(cgpa) FROM students WHERE dept = s.dept);
-- percentage of the total
SELECT name, credits * 100 / (SELECT SUM(credits) FROM students)
  AS pct FROM students;
-- count by condition (shows 0 too)
SELECT SUM(cgpa >= 3.5) AS top, SUM(cgpa < 3.5) AS rest
FROM students;
-- top 2 dates by total
SELECT joined, SUM(credits) AS t FROM students
GROUP BY joined ORDER BY t DESC LIMIT 2;
```

Question 1 · JavaScript

Spring 2026 (261) Set A

index.html

```
<!DOCTYPE html>
<html><head><title>Study Tracker</title></head><body>
Subject Name: <input type="text" id="subject"><br><br>
Study Hours: <input type="number" id="hours"><br><br>
<button onclick="addStudy()">Add Study</button><br><br>
<label id="report"></label>
<script src="script.js"></script></body></html>
```

script.js

```
const GOAL = 20;      // a. weekly goal
let total = 0;        // b. running total
let subjects = [];    // c. subject names
let sessions = 0;     // d. number of sessions

function addStudy() {
  let subject =
    document.getElementById("subject").value.trim();
  let hours =
    parseFloat(document.getElementById("hours").value);
  if (subject === "" || isNaN(hours) || hours < 0) {
    document.getElementById("report").innerHTML =
      "Enter a subject and valid hours.";
    return;
  }
  total += hours;                    // b
  subjects.push(subject);            // c
  sessions++;                        // d
  let average = total / sessions;    // e
  average = Math.round(average * 1000) / 1000;

  let feedback;                      // g: test from the top
  if (total >= GOAL) feedback = "Weekly goal reached!";
  else if (total > 12) feedback = "Almost ready!";
  else if (total > 5) feedback = "Good progress!";
  else feedback = "Start studying regularly!";
  if (sessions > 7 && total < GOAL)  // h
    feedback = "Increase your study time!";

  document.getElementById("report").innerHTML =    // f
    `Total: ${total} hours, Average: ${average} hours, ` +
    `Subjects: [${subjects.join(", ")}], ${feedback}`;
}
```

Question 2 · PHP

Spring 2026 (261) Set A

index.php

```
<!DOCTYPE html><html><body>
<form method="POST" action="">
  Credits/course:
  <input type="number" name="credits" required><br>
  Courses completed:
  <input type="number" name="courses" required><br>
  Target credits:
  <input type="number" name="target" required><br>
  <input type="submit" name="calc" value="Calculate">
</form>
<?php
if (isset($_POST["calc"])) {
  $credits = (int) $_POST["credits"];
  $courses = (int) $_POST["courses"];
  $target = (int) $_POST["target"];
  $total = $credits * $courses;          // credits earned

  if ($total >= 120)     $progress = "Excellent";
  elseif ($total >= 90)  $progress = "Good";
  elseif ($total >= 60)  $progress = "Average";
  else                   $progress = "Poor";

  $diff = $total - $target;              // vs the target
  if ($diff > 0)
    $result = "Above target by $diff";
  elseif ($diff < 0)
    $result = "Below target by " . abs($diff);
  else
    $result = "Target met exactly (0 difference)";

  echo "<table border='1'><tr><th>Total Credits</th>";
  echo "<th>Progress</th><th>Target</th><th>Result</th></tr>";
  echo "<tr><td>$total</td><td>$progress</td>";
  echo "<td>$target</td><td>$result</td></tr></table>";
}
?>
</body></html>
```

Question 3 · PHP & MySQL

Spring 2026 (261) Set A

tourism.sql

```
CREATE DATABASE tourism;
USE tourism;
CREATE TABLE tourist_spot (
  SpotID INT PRIMARY KEY,
  SpotName VARCHAR(100),
  Region VARCHAR(10),
  Category VARCHAR(30),
  Rating DECIMAL(3,1),         -- exact decimals (not FLOAT)
  EntryFee DECIMAL(8,2),
  VisitorsPerYear INT
);
INSERT INTO tourist_spot VALUES
(1, 'Cox''s Bazar Beach', 'CTG', 'Beach', 4.9, 0, 150000),
(2, 'Sundarbans Forest', 'KHL', 'Nature', 4.3, 150, 80000),
(3, 'Ratargul Forest', 'SYL', 'Nature', 4.8, 100, 60000),
(4, 'Kuakata Beach', 'PTU', 'Beach', 4.5, 0, 90000),
(5, 'Shat Gambuj Mosque', 'BGH', 'Heritage', 4.7, 30, 70000),
(6, 'Paharpur', 'NWG', 'Heritage', 4.1, 30, 40000);
```

index.php

```
<?php
$conn = new mysqli("localhost", "root", "", "tourism");
if ($conn->connect_error)
  die("Connection failed: " . $conn->connect_error);

function show($conn, $sql) {   // prints a SELECT as a table
  $res = $conn->query($sql);
  echo "<table border='1'><tr>";
  foreach ($res->fetch_fields() as $f)
    echo "<th>$f->name</th>";
  echo "</tr>";
  while ($row = $res->fetch_assoc()) {
    echo "<tr>";
    foreach ($row as $v)
      echo "<td>" . htmlspecialchars($v) . "</td>";
    echo "</tr>";
  }
  echo "</table>";
}
?>
<!DOCTYPE html>
<html><body>
<h3>1) Rating above 4.5 or name contains Beach</h3>
<?php show($conn, "SELECT SpotName, Region, Rating
  FROM tourist_spot
  WHERE Rating > 4.5 OR SpotName LIKE '%Beach%'
  ORDER BY Rating DESC"); ?>

<h3>2) Rating +0.2 and EntryFee +10%</h3>
<form method="POST" action="">
  <button type="submit" name="task2">Run task 2</button>
</form>
<?php
if (isset($_POST["task2"])) {
  $conn->query("UPDATE tourist_spot
    SET Rating = Rating + 0.2, EntryFee = EntryFee * 1.10
    WHERE Rating BETWEEN 4.0 AND 4.5 AND EntryFee > 0");
  echo "<p>" . $conn->affected_rows . " spot(s) updated.</p>";
}
show($conn, "SELECT * FROM tourist_spot");
?>

<h3>3) Categories with average rating above 4.4</h3>
<?php show($conn, "SELECT Category, COUNT(*) AS Spots,
  ROUND(AVG(Rating), 2) AS AvgRating,
  SUM(VisitorsPerYear) AS TotalVisitors
  FROM tourist_spot GROUP BY Category
  HAVING AVG(Rating) > 4.4 ORDER BY AvgRating DESC"); ?>

<h3>4) Regions with revenue above 1,000,000</h3>
<?php show($conn, "SELECT Region,
  SUM(EntryFee * VisitorsPerYear) AS Revenue
  FROM tourist_spot GROUP BY Region
  HAVING Revenue > 1000000 ORDER BY Revenue DESC");
$conn->close(); ?>
</body></html>
```